STCS I
Fundamentals of Data Mining

Lecture 6: Text Mining

Week 9

1

Learning Objectives

- By the end of this lecture, students should be able to:
  - Explain what text mining is and how it relates to NLP, IR, and IE
  - Describe the KDD process for text mining
  - Apply key text processing steps (tokenisation, stop-word removal, normalisation)
  - Represent documents using BoW, Counts, Boolean, TF, IDF, TF-IDF and vector models
  - Calculate and interpret cosine similarity between documents (& queries)
  - Compare classical and modern text mining approaches
  - Recognise evaluation, ethical & practical issues in text mining systems.

2

The business opportunity in text mining

- Over **80% of enterprise data** in **unstructured** form, e.g. in emails, documents, customer feedback etc. that include text data, audio, images etc.
- Extract **actionable insights** for:
  - customer experience
  - risk management
  - competitive intelligence
  - product innovation etc.
- Global data creation:
  - **>120 zettabytes** in 2023 (Statista, 2024)
  - projected to exceed 180 zettabytes by 2025.

3

Text Mining - Intro

- Text mining studies **how to extract useful patterns, structure, and decisions from text** data.
- Text is (unstructured) data
  - must be represented in numerical form for algorithms
  - meaning is communicated through language (not tables)
  - values comes from turning text into analysable representations.
- Needs text pre-processing & feature extraction
- The field evolved from keyword count (statistical methods) to language modelling (contextual neural models and LLMs).

4

Why text mining matters

- Large volumes of organisational data are text:
  - emails, feedback, forms, reports, social media, articles
- Text mining supports:
  - search,
  - monitoring,
  - classification,
  - clustering,
  - extraction,
  - summarisation,
  - question answering.
- Foundational to search engines, recommender systems, chatbots, and LLM-based tools
- E.g.s: student feedback, health reports, government memos

5

What text mining is?

- Text mining is:
  - The **discovery of useful patterns**, **structure**, or **predictions** from text collections (i.e. corpus)
- Major sub-areas of Text mining:
  - **Natural Language Processing** (NLP): methods for processing and modelling language.
  - **Information Retrieval** (IR): retrieving and ranking relevant documents for a query.
  - **Information Extraction** (IE): extracting structured facts from text.
- Progress to language modelling e.g. LLM

6

What text mining is?
Natural Language Processing (NLP)

- **Objective**:
  - Enable machines to **understand**, **interpret**, and **generate** human language
- **Why NLP?**
  - Human language is **unstructured** and **ambiguous**; computers need structured input
- **Key tasks:**
  - Tokenisation: split text into words
  - POS (Parts-of-Speech) tagging: assign word classes e.g. nouns, verbs
  - NER (Named Entity Recognition): identify names, places, etc.
  - Parsing: analyse sentence structure
  - Sentiment: detect emotions/opinions
- **Applications**
  - Chatbots, translation, text summarisation, search, speech recognition

7

7

Historical Evolution of Text mining

- Stage of evolution:
  1. Rule-based and keyword systems
  2. Statistical IR and bag-of-words
  3. Classical ML with sparse vectors
  4. Distributed word embeddings
  5. Deep contextual models
  6. Large Language Models (LLMs)
- Each stage improve representation:
  - Exact keywords → Weighted words →..
    →Learned vector representation → Contextual meaning →..
    → Generative reasoning over text

8

8

Text mining KDD

1. **Domain Understanding**
   - Define task: classification, similarity, retrieval, summarization, etc.
2. **Data Selection**
   - Collect relevant documents (and terms) from **corpus** (e.g., feedback, emails, reports)
3. **Data Cleaning & Preprocessing**
   - Tokenisation, stopword removal, normalisation, etc.
4. **Data Transformation**
   - Bag-of-Words, Boolean, TF-IDF, Vector Space Model, Embeddings
5. **Data Mining**
   - Apply text mining algorithms to perform task in (1)
6. **Evaluation & Interpretation**
   - – Accuracy, similarity scores, relevance, usefulness, etc.
7. **Knowledge Presentation & Use**
   - Decision support, search systems, dashboards, AI tools

9

9

Text mining KDD

1. **Domain Understanding** - What problem are we solving?
   - Define task: classification, similarity, retrieval, summarization, etc.
2. **Data Selection** - What text do we use?
   - Collect relevant documents & terms from corpus (e.g., feedback, emails, reports)
3. **Data Cleaning & Preprocessing** - How to prepare the raw text?
   - Tokenisation, stopword removal, normalisation, etc.
4. **Data Transformation** - How do we convert text into features?
   - Bag-of-Words, Boolean, TF-IDF, Vector Space Model, Embeddings
5. **Data Mining** - What patterns or predictions do we learn?
   - Apply text mining algorithms to perform task in (1)
6. **Evaluation & Interpretation** - How good are the results?
   - – Accuracy, similarity scores, relevance, usefulness, etc.
7. **Knowledge Presentation & Use** - How is the result used?
   - Decision support, search systems, dashboards, AI tools

10

10

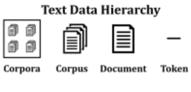
Text Mining
First steps in KDD

- **Step 1: Domain understanding**
  - What problem are we solving? Insight/outcome needed from data?
  - Define task: classification? Similarity?, retrieval? Summarisation? etc.
- **Step 2: Data Selection**
  - What text do we use? Data must be relevant to the task
  - Collect from relevant sources: e.g. survey feedback, emails, reports, news, posts, etc.
- **Terminology**
  - Corpus is a collection of documents used for analysis
  - Corpora is a plural of corpus
  - Document is a single record of text (e.g. one comment, one email, etc.)
  - Word is a basic unit of text, transformed to token after tokenisation

11

11

Our Example:
Student Feedback



https://botpenguin.com/glossary/corpus

- Start with data available – i.e. the corpus
- A **corpus** is a collection of text documents used for analysis, modelling, or learning patterns in language.
- Here, corpus is all the comments from a *Student Feedback survey* on a course.
- Sample of 5 student feedback (comments):
  1. *The lectures are clear and very helpful for understanding concepts*
  2. *The lectures have too much theory and are Not Very Practical*
  3. *The labs are helpful and very practical for understanding concepts*
  4. *The Lectures are clear but the pace of lectures is TOO FAST for concepts*
  5. *The practical labs are very helpful and engaging for students*

12

12

Core tasks in text mining

- Tasks can be *predictive*, *descriptive* or *retrieval* oriented
- **Predictive tasks** (label or decision)
  - **Document Classification**
    e.g.: *Will this student feedback be positive or negative?*
  - **Sentiment Detection**
    e.g.: *Is this comment supportive or critical of the course?*
- **Descriptive tasks** (discover patterns or structure)
  - **Clustering / Topic Discovery**
    e.g.: *What groups or themes exist in the feedback comments?*
  - **Information Extraction**
    e.g.: *What key entities or issues (e.g. lecturer, pace, labs) are mentioned?*
  - **Summarisation**
    e.g.: *What is the main summary of all student feedback?*

Core tasks in text mining (cont..)

- **Retrieval** tasks (find relevant information – use "query")
  - **Search and Ranking**
    E.g.: *Which comments are most relevant to "practical sessions"?*
  - **Semantic Similarity**
    E.g.: *Which feedback comments are most similar to this one?*
  - **Question Answering**
    E.g.: *What do students say about the pace of lectures?*

- Example *Student Feedback* problem:
  - *"Determine how similar student feedback comments are, in order to identify common themes and patterns in course feedback."*
  - **Task type:** Descriptive / Retrieval
- Task type determines *representation*, *algorithm* and *evaluation*!

Text mining KDD

1. **Domain Understanding** - What problem are we solving?
   - Define task: classification, similarity, retrieval, summarization, etc.
2. **Data Selection** - What text do we use?
   - Collect relevant documents & terms from **corpus** (e.g., feedback, emails, reports)
3. **Data Cleaning & Preprocessing** - How to prepare the raw text?
   - Tokenisation, stopword removal, normalisation, etc.
4. **Data Transformation** - How do we convert text into features?
   - Bag-of-Words, Boolean, TF-IDF, Vector Space Model, Embeddings
5. **Data Mining** - What patterns or predictions do we learn?
   - Apply text mining algorithms to perform task in (1)
6. **Evaluation & Interpretation** - How good are the results?
   - – Accuracy, similarity scores, relevance, usefulness, etc.
7. **Knowledge Presentation & Use** - How is the result used?
   - Decision support, search systems, dashboards, AI tools

Feature Selection:
Why it matters?

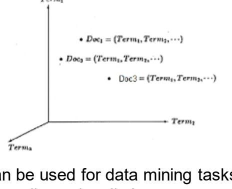
- **Feature selection** - process of identifying and keeping only the most **relevant features** (terms) from a collection (of documents) to build effective models.

- Why select features?
  - Text data is **high dimensional** e.g., 10K+ unique words
  - Many features or terms are **irrelevant** or **noisy**
  - **Not all** features **contribute meaningfully** to task e.g. classification

- Feature selection improves model efficiency, accuracy & interpretability

Feature Selection:
Features become vectors

- Each document now a high dimensional vector:
  - Doc 1 → feature vector = ($Term_1$, $Term_2$,…, $Term_n$)
  - Doc 2 → feature vector = ($Term_1$, $Term_2$,…, $Term_n$)



- Vectors can be used for data mining tasks e.g. clustering, BUT first – reduce dimensionality!

Feature Selection:
Dimensionality Reduction

- Text mining algorithms are optimised when only a subset of the discriminative terms are selected
  - Even after stemming and stopword removal.

- **Feature Selection Methods**:
  1. **Greedy search**
     - Start from full set and delete one term at a time
     - Eliminate least important term (e.g. using Gini index)
  2. **Dimensionality reduction**
     - Use Principal Component Analysis (PCA), Singular Value Decomposition (SVD), or clustering
     - Groups correlated features
     - Produces low-rank matrix (smaller feature space)
- At times performance will not improve even with massive reductions
  - E.g. In a fraud case, 140 out of 20,000 terms still gave accurate results.

Text mining KDD

1. **Domain Understanding** - What problem are we solving?
   – Define task: classification, similarity, retrieval, summarization, etc.
2. **Data Selection** - What text do we use?
   • Collect documents from **corpus** (e.g., feedback, emails, reports)
3. **Data Cleaning & Preprocessing** - How to prepare the raw text?
   – Tokenisation, stopword removal, normalisation, etc.
4. **Data Transformation** - How do we convert text into features?
   – Bag-of-Words, Boolean, TF-IDF, Vector Space Model, Embeddings
5. **Data Mining** - What patterns or predictions do we learn?
   – Apply text mining algorithms to perform task in (1)
6. **Evaluation & Interpretation** - How good are the results?
   – – Accuracy, similarity scores, relevance, usefulness, etc.
7. **Knowledge Presentation & Use** - How is the result used?
   – Decision support, search systems, dashboards, AI tools

19

19

Text Pre-processing

- **Depends on the task**
  – E.g. summarise, search, sentiment, topic discovery, LLM prompting do not all require the same cleaning
- **Text preprocessing tasks**:
  – Case handling (lowercasing)
  – Tokenisation
  – Stopword handling (*noise removal*)
  – Stemming vs lemmatisation (*normalisation*)
  – Handling punctuation, numbers, emojis, URLs
  – Domain-specific cleaning
  – Optional phrase extraction / n-grams

20

20

Text Pre-processing: *Lowercasing*

- Original documents (comment ≡ document):

Doc	Feedback Text
D1	The lectures are clear and very helpful for understanding concepts
D2	The lectures have too much theory and are Not Very Practical
D3	The labs are helpful and very practical for understanding concepts
D4	The Lectures are clear but the pace of lectures is TOO FAST for students
D5	The practical labs are very helpful and engaging for students

- ***Lowercasing*** – convert to lowercase to reduce vocabulary size.

Doc	Feedback Text
D1	the lectures are clear and very helpful for understanding concepts
D2	the lectures have too much theory and are not very practical
D3	the labs are helpful and very practical for understanding concepts
D4	the lectures are clear but the pace of lectures is too fast for students
D5	the practical labs are very helpful and engaging for students

21

21

Text Pre-processing: *Tokenisation*

- ***Tokenisation*** – split into smaller discrete units (tokens), often words.

Doc	Tokens
D1	[the, lectures, are, clear, and, very, helpful, for, understanding, concepts]
D2	[the, lectures, have, too, much, theory, and, are, not, very, practical]
D3	[the, labs, are, helpful, and, very, practical, for, understanding, concepts]
D4	[the, lectures, are, clear, but, the, pace, of, lectures, is, too, fast, for, students]
D5	[the, practical, labs, are, very, helpful, and, engaging, for, students]

22

22

Text Pre-processing: *Stopword removal*

- ***Stopword Removal*** – Remove very common words (e.g. "*the*", "*and*", "*is*"), these carry little meaning and create noise.

Doc	Tokens (After Stopword Removal)
D1	[lectures, clear, helpful, understanding, concepts]
D2	[lectures, too, much, theory, not, practical]
D3	[labs, helpful, practical, understanding, concepts]
D4	[lectures, clear, pace, lecture, fast, students]
D5	[practical, labs, helpful, engaging, students]

23

23

Text Pre-processing: Normalisation (*Stemming vs Lemmatisation*)

- Stemming and lemmatisation both reduce words to a common base form – i.e. normalisation
- **Aim**: improve similarity matching and reduce vocabulary size.
  – E.g. so "*lecture*" and "*lectures*" are treated the same
- **Stemming** (Rule-based)
  – Cuts words mechanically
  – E.g. lectures →lectur; engaging/engagement →engag
  – Fast but rough
- **Lemmatisation** (Dictionary-based)
  – Uses vocabulary and grammar
  – E.g. lectures → lecture; engaging/engagement →engage
  – Slow but accurate and preferred in NLP

24

24

Text Pre-processing:
Stemming vs Lemmatisation (cont..)

- Words stemmed and lemmatised

Original Word	Stemmed Form	Lemmatised Form
lectures	lectur	lecture
understanding	understand	understand
concepts	concept	concept
helpful	help	helpful
practical	practic	practical
engaging	engag	engage
students	student	student

25

Other Text Pre-processing

- Additional Text Pre-processing Tasks

Task	What it involves	Why do?	Example
Handle punctuation, numbers, emojis, URLs	Remove or normalise symbols, numbers, links, emojis	These often add noise and do not help most modelling tasks	"The lectures are great!!! 😊" →"lectures great"
Domain-specific cleaning	Apply rules specific to the dataset/ domain	Removes irrelevant or misleading terms specific to context	"student 202610011 registered late" →remove "202610011" (not meaningful)
Phrase extraction / n-grams	Combine words into phrases (e.g. 2-word or 3-word sequences)	Captures meaning lost in single words	"not practical" is a bigram: (important negative meaning)

26

Text mining KDD

1. **Domain Understanding -** What problem are we solving?
   – Define task: classification, similarity, retrieval, summarization, etc.
2. **Data Selection -** What text do we use?
   • Collect documents from **corpus** (e.g., feedback, emails, reports)
3. **Data Cleaning & Preprocessing - How to prepare the raw text?**
   • Tokenisation, stopword removal, normalisation, etc.
4. **Data Transformation -** How do we convert text into features?
   – Bag-of-Words, Boolean, TF-IDF, Vector Space Model, Embeddings
5. **Data Mining -** What patterns or predictions do we learn?
   – Apply text mining algorithms to perform task in (1)
6. **Evaluation & Interpretation -** How good are the results?
   – – Accuracy, similarity scores, relevance, usefulness, etc.
7. **Knowledge Presentation & Use -** How is the result used?
   – Decision support, search systems, dashboards, AI tools

27

Text Representation

- *Converts raw text into numerical form* (features/vectors) for algorithms.
  – sentences need to be converted to numbers
- Text is **unstructured** and **ambiguous** - representation gives structure
- *Similarity* and *learning* depend on representation
  – different representations will give different results!
- **A good representation** should capture:
  – *Importance* of words or features
  – *Relationships* between documents
  – *Meaning* and context (advanced)
- Representation **methods** – Bag-of-Words (BoW), TF-IDF (weighted vectors), embedding (dense vectors)

28

Text Representation
Bag-of-Words Model

- A Bag-of-Words (BoW) is a method that represents each document as a vector of word frequencies (presence).
- A BoW is based on a fixed vocabulary of terms in the corpus
- **Vocabulary** is all selected terms in the corpus
- **Document** represented as a **vector** over the vocabulary
- Word order and grammar do not matter in BoW!
- Similar to *feature matrices*
  - o Features ≡ words
  - o Records ≡ documents
  - o Values ≡ Boolean (0/1), counts (frequency), or weights (e.g. TF-IDF)

29

Text Representation
Bag-of-Words (Example)

- **Term Frequency** (TF)
  – Count of how many times each term in the vocabulary appears in a document
  – Forms a vector: *Doc=(Term1_count, Term2_count,…)*
- The BoW model
  – Lists unique terms across all documents (in vocabulary)
  – Gets TF for all terms in each document
  – *Outcome*: Transforms text into numerical vectors for machine learning
- Our example (pre-processed text):
  – *D1: lecture, clear, helpful, understand, concept*
  – *D4: lecture, clear, pace, lecture, fast, student*
  – *D5: practical, lab, helpful, engage, student*

Doc	*clear*	*concept*	*engage*	*helpful*	*lab*	*lecture*	*practical*	*student*	*understand*
D1	1	1	0	1	0	1	0	0	1
D4	1	0	0	0	0	2	0	1	0
D5	0	0	1	1	1	0	1	1	0

30

Text Representation
Bag-of-Words:Term vectors

- In BoW, each *document* is represented as a *term vector*
- In general, a **vector** is a list of values recorded in a consistent order
- In a term vector, each position corresponds to a unique term or word in the dictionary
- Term vectors values can be **binary** (0/1) indicating presence, or **counts** for number of times term appears in document.
- Terms = [*clear, concept, engage, helpful, lab, lecture, practical, student, understand*]
- Our example (Counts):
  - D1 = [1, 1, 0, 1, 0, 1, 0, 0, 1]
  - D4 = [1, 0, 0, 0, 0, 2, 0, 1, 0]
  - D5 = [0, 0, 1, 1, 1, 0, 1, 1, 0]
- If term values are Boolean then:
  - D4 = [0, 0, 0, 0, 0, 1, 0, 1, 0] since "lecture" appears 2x in doc D4 but only present counts in this representation

31

31

Text Representation
Bag-of-Words: Challenges

https://medium.com/@vamshiprakash001/an-introduction-to-bag-of-words-bow-c32a65293ccc

- Large and sparse vectors due to large vocabulary
- *Domain dependence* create own common words in each dataset.
  - E.g. "virus, "covid", "rona", in twitter on Covid-19.
  - Domain related common words can be treated as stopwords.
- Raw word counts, i.e. term frequency (TF) not always meaningful
  - **High count ≠ High Importance**
- *Context is ignored:*
  - ***Word order***, ***meaning*** and ***part-of-speech*** are lost
  - E.g. "bank" – river or finance? Ambiguous without context

32

32

Text Representation:
Boolean

- Boolean model uses **binary representation only** (i.e. 0 or 1 for presence or absence)
- Build **queries** by combining terms with Boolean operators:
  - AND, OR, NOT
  - The system returns all documents that satisfy the query
- Example of document retrieval:

Doc	*clear*	*concept*	*engage*	*helpful*	*lab*	*lecture*	*practical*	*student*	*understand*
D1	1	1	0	1	0	1	0	0	1
D4	1	0	0	0	0	2	0	1	0
D5	0	0	1	1	1	0	1	1	0

  - **Query 1:** *"lecture* OR *helpful"* → D1, D4, D5 (3 documents retrieved)
  - **Query 2:** *"clear* AND *concept* AND *understand"* → D1
  - **Query 3:** *"lab* AND *practical"* → D5**;**
  - **Query 4:** *"clear* NOT *concept"* → (empty set)

33

33

Text Representation:
Boolean – Strengths vs Limits

- **Strengths**
  - simple and clean mathematical formalism
    - Uses basic logical operators AND, OR, NOT
  - binary term weighting (either 1 or 0)
    - Term in document either present (1) or absent (0)
    - No need to calculate term frequencies
- **Limitations**
  - no partial match if using Boolean logics
    - Doc must match full query or not at all
  - no encoding of frequency
    - Ignores how often term appears in doc
  - no ranking
    - Docs treated equally, no relevance score

34

34

Text Representation:
TF, IDF, TF-IDF

- **Term Frequency** (TF):
  - Number of many times a term occurs in a document. Measures importance of the term within the document
- **Document Frequency** (DF):
  - How many documents contain the term. Measures how common a term is across documents
- **Inverse Document Frequency** (IDF):
  - Down-weights common terms across the dictionary. Combines local importance with global rarity

$$IDF(t) = \log\left(\frac{N}{DF(t)}\right)$$

  - Where t is the term; N is total number of documents in the corpus and $DF(t)$ is the number of documents in N containing the term t

35

35

Text Representation:
TF-IDF

- **TF-IDF** is a **weight** given to each term in each document. Reflects importance of term ***relative to the corpus***.
- Combines TF and IDF:

$$TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t)$$

  - where t is a term, d is a document, and $\mathrm{TF}(t,d)$ is the frequency of the term t in document d
- IDF measures how rare a term is across documents
- **Important words** are:
  - frequent in a document but rare across the corpus (i.e. full collection of documents)
- TF-IDF **improves BoW** by assigning weights to terms based on their importance.

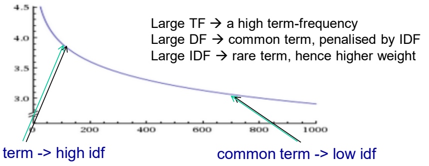
36

36

Calculating TF-IDF Term Weighting

$$w(t) = \mathrm{tf} \cdot \log \frac{N}{n}$$

w weight assigned to term *t* in document
tf number of occurrence of term *t* in document
N number of documents in entire collection
n number of documents with term *t*



N = 806,791
(Reuters collection)

Text Representation:
TF-IDF *(Our Example)*

Exercise: Let's calculate TF-IDF for document **D** within the corpus:

- TF – almost all terms appear only once in the document
- IDF – down-weight frequently occurring terms

D4	clear	Concept	engage	helpful	lab	lecture	practical	student	understand
TF	1	0	0	0	0	2	0	1	0
DF	2	2	1	3	2	4	3	2	2
IDF	0.398	0.398	0.699	0.222	0.398	0.097	0.222	0.398	0.398
TF-IDF	0.398	0	0	0	0	0.194	0	0.398	0

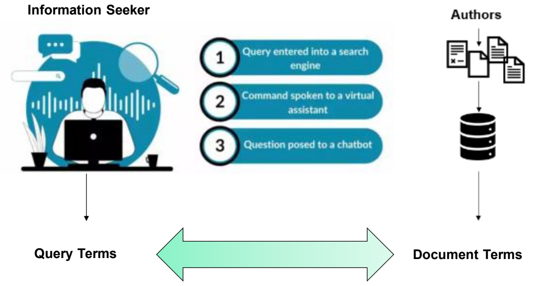
- Rare terms across documents give higher importance
- *"lecture"* is frequent but **not distinctive**
- *"clear"* and *"student"* are less common and **more discriminative**

Text Representation:
TF-IDF *(Example in Query Processing)*

- Simplistic ***search engine query-based retrieval*** example
- Given a corpus of news stories involving countries, economic issues, and espionage activities:
  - The training set *D* with 6 documents
    - d_1={GERMAN, PW}
    - d_2={US, US, ECONOM, SPY}
    - d_3={US, BILL, ECONOM, ESPIONAG}
    - d_4={US, ECONOM, ESPIONAG, BILL}
    - d_5={GERMAN, MAN, VW, ESPIONAG}
    - d_6={GERMAN, GERMAN, MAN, PW, SPY}
- A user submits a query q with the terms:

  q = {US, ECONOM, BILL, SPY, ESPIONAG}

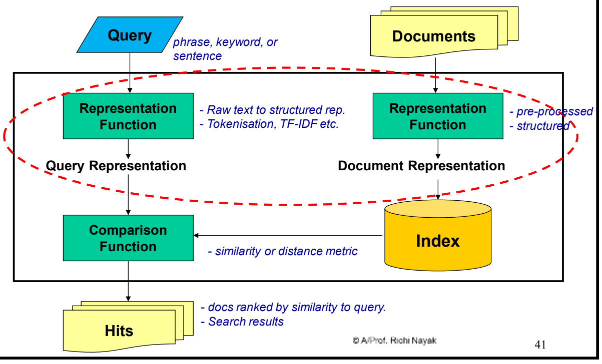
Query Processing:
Information Retrieval



Do these represent the same concepts?

Image: https://spotintelligence.com/2024/04/03/query-understanding/

Query Processing:
Representing Text for Query Processing



Measuring Similarity:
TF-IDF *(Another Example)*

- The terms "*spy*" and "*bill*" have higher relative weights
- Docs with these terms are more likely to be retrieved and ranked higher due to their TF-IDF
- No doc contains both terms, so d_2, d_6 (containing "spy") and d_3, d_4 (containing "bill") will be retrieved.

Term in q	tf	df	tf-idf (weight)
US	1	3	1*log(6/3)=0.301
ECONOM	1	3	1*log(6/3)=0.301
BILL	1	2	1*log(6/2)=0.477
SPY	1	2	1*log(6/2)=0.477
ESPIONAG	1	3	1*log(6/3)=0.301

Text Representation
Embeddings (Dense Vectors)

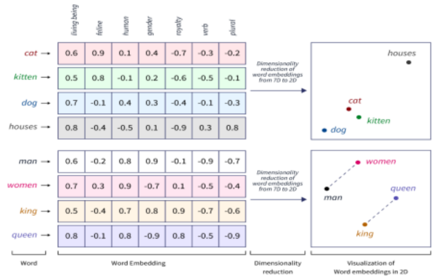
- **Embeddings** represent words, sentences, or documents as dense numerical vectors
- Learned from data / context
- Improves BoW & TF-IDF by adding **compositional meaning**
- Words with similar meanings have similar vector representations
  - "lecture" ≈ "class"
  - "helpful" ≈ "useful", whereas in BoW "helpful" ≠ "useful" (no match)

- **Types of embeddings**:
  - *Word embeddings* – one vector per word (e.g. Word2Vec model)
  - *Sentence / document embedding* – one vector per text

43

43

Text Representation
Word Embeddings - Example



https://www.scaler.com/topics/tensorflow/tensorflow-word-embeddings/

SCALER Topics

44

44

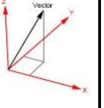
Text Representation
Towards Vector Space Models (VSM)

- **BoW and TF-IDF limitations**
  - No word order – e.g. "not practical" ≈ "practical"
  - No compositional meaning - cannot combine for full meaning
  - Weak handling of synonymy and polysemy
    - "helpful" ≠ "useful"
    - "bank" (river vs finance) treated the same
  - Sparse high-dimensional vectors -
    - many zeros, inefficient representation
- **Motivation for Embeddings** (Dense Vectors)
  - Move from ***word matching*** to ***meaning matching***
  - Similar meanings → closer in vector space
- **<u>All</u> representations map text to vectors, which exist in vector space!**
- **SIMILARITY (e.g. cosine) compares documents in this space!**

45

45

From Vectors to Similarity



- Once documents are represented as vectors, we can **measure how similar documents are**
- ***Text vectors***:
  - Each document → a vector in space
  - Similar documents → **vectors close together**
  - Dissimilar documents → vectors far apart
- **Similarity contexts**:
  - *Document similarity* – compare documents as vectors
  - *String similarity* – compare sequence of characters (e.g. edit distance)
  - *Lexical similarity* – compare documents based on shared words (terms). Uses BoW / TF-IDF + Cosine similarity
  - *Semantic similarity* – compare meaning, not exact words (uses embeddings)

46

46

Measuring Similarity:
Typical Distance Metrics

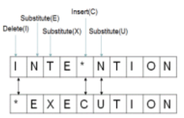
- **Cosine Distance**

$$d(x,y) = 1 - cosine(x,y)$$

  - measures angle between vectors
  - Common in text mining for high dimensional sparse vectors
  - Values range from 0 (similar) to 1 (dissimilar)
- **Minkowski Distance**

$$d(x,y) = \sqrt[p]{\sum_{i=1}^{V}(x_i - y_i)^p}$$

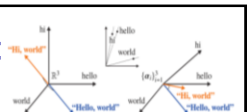
  - when p=2, it is Euclidean Distance
  - Used for numerical feature spaces
- **Edit Distance (Levenshtein Distance)**
  - Min. number of operations (e.g. insertions, deletions, substitution) to convert one string to another
  - E.g. *"INTENTION"* → *"EXECUTION"* *edit distance 5



47

47

Measuring Similarity:
Cosine Similarity



- Cosine similarity is the most **common similarity measure**. It <u>measures the angle between vectors</u>.
- Focuses on **direction** (not affected by document length)

$$\mathrm{sim}(d_1, d_2) = \frac{d_1 \cdot d_2}{\|d_1\| \|d_2\|}$$

  - where d_1 and d_2 are doc vectors, and $sim(d_1, D_2)$ is a measure of their similarity according to angle between them
- Range is [0,1] given cosine θ
  - $\mathrm{Cos}(0^o)$ = 1 → documents are identical
  - $\mathrm{Cos}(90^o)$ = 0 → no similarity (orthogonal)
- Works well for sparse, high-dimensional vectors (BoW, TF-IDF)

48

48

Measuring Similarity:
Cosine Similarity (Example)

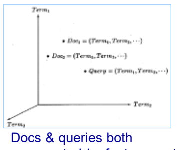
- Say we have a corpus of news stories involving countries, economic issues, and espionage activities.
- The training set D with 6 documents
  - d_1 = {GERMAN, PW}
  - d_2 = {US, US, ECONOM, SPY}
  - d_3 = {US, BILL, ECONOM, ESPIONAG}
  - d_4 = {US, ECONOM, ESPIONAG, BILL}
  - d_5 = {GERMAN, MAN, VW, ESPIONAG}
  - d_6 = {GERMAN, GERMAN, MAN, PW, SPY}
- A user submits a query q with the terms:
  - q = {US, ECONOM, BILL, SPY, ESPIONAG}
- You want to compute query-document similarity.

49

49

Measuring Similarity:
Cosine Similarity (Example)

- Task: Query-document similarity
  - Identify which docs are relevant and match best to query
- Query-term space:
  - Q = {US, ECONOM, BILL, SPY, ESPIONAG}
- Document vectors (with term weightings from **TF-IDF**):
  - d_1=[0,0,0,0,0]
  - d_2=[0.602,0.301,0,0.477,0]
  - d_3=[0.301,0.301,0.477,0,0.301]
  - d_4=[0.301,0.301,0.477,0,0.301]
  - d_5=[0,0,0,0,0.301]
  - d_6=[0,0,0,0.477,0]
  - Q=[0.301,0.301,0.477,0.477,0.301]



Docs & queries both represented by feature vectors

50

50

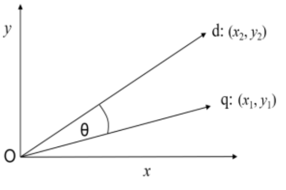
Cosine Similarity
(query, document)

Given two vectors:

- **Query** vector: $\mathbf{q} = (x_1, y_1)$
- **Document** vector: $\mathbf{d} = (x_2, y_2)$

The **cosine similarity** is defined as:

$$\cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \cdot \|\mathbf{d}\|}$$



Where:

- $\mathbf{q} \cdot \mathbf{d}$ = dot product = $x_1 x_2 + y_1 y_2$
- $\|\mathbf{q}\| = \sqrt{x_1^2 + y_1^2}$
- $\|\mathbf{d}\| = \sqrt{x_2^2 + y_2^2}$

The **cosine distance** (used in retrieval) is then:

$$d(\mathbf{q}, \mathbf{d}) = 1 - \cos(\theta) = 1 - \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \cdot \|\mathbf{d}\|}$$

51

51

Cosine Similarity (Example)

- Cosine similarity between Query Q and doc d_2

$$\mathrm{sim}(q, d_2) = \frac{q \cdot d_2}{\|q\| \|d_2\|}$$

Step 1: Dot product

$$q \cdot d_2 = (0.301)(0.602) + (0.301)(0.301) + (0.477)(0) + (0.477)(0.477) + (0.301)(0)$$

$$= 0.181 + 0.091 + 0 + 0.228 + 0 = 0.500$$

Step 2: Magnitudes

$$\|q\| = \sqrt{0.301^2 + 0.301^2 + 0.477^2 + 0.477^2 + 0.301^2} \approx 0.853$$

$$\|d_2\| = \sqrt{0.602^2 + 0.301^2 + 0.477^2} \approx 0.825$$

Step 3: Cosine similarity

$$\mathrm{sim}(q, d_2) = \frac{0.500}{0.853 \times 0.825} \approx \frac{0.500}{0.704} \approx \boxed{0.71}$$

Exercise – Do similarity between Q and d_4 – which doc more similar to Q?

52

52

Term Vector Space Representation:
Strengths & Limitations

- **Strengths**:
  - supports **term weighting** (e.g. TF-IDF)
  - allows **partial matching** of terms, unlike the boolean model
  - allows for **ranking**: sort in decreasing values of similarity
  - applies document **length normalisation** (to handle document size variation
- **Limitations:**
  - treats terms **independently** (no context or semantics)
  - fails to capture **relationships** between terms or **word order**

53

53

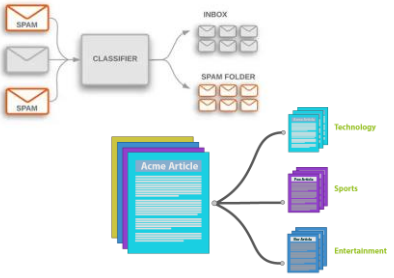
Text mining KDD

1. **Domain Understanding -** What problem are we solving?
   - Define task: classification, similarity, retrieval, summarization, etc.
2. **Data Selection -** What text do we use?
   - Collect documents from **corpus** (e.g., feedback, emails, reports)
3. **Data Cleaning & Preprocessing - How to prepare the raw text?**
   - Tokenisation, stopword removal, normalisation, etc.
4. **Data Transformation - How do we convert text into features?**
   - Bag-of-Words, Boolean, TF-IDF, Vector Space Model, Embeddings
5. **Data Mining -** What patterns or predictions do we learn?
   - Apply text mining algorithms to perform task in (1)
6. **Evaluation & Interpretation -** How good are the results?
   - – Accuracy, similarity scores, relevance, usefulness, etc.
7. **Knowledge Presentation & Use -** How is the result used?
   - Decision support, search systems, dashboards, AI tools

54

54

Text Mining Tasks:
Classification

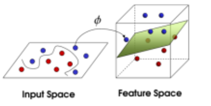


55

55

Text Mining Tasks:
Some Classification Algorithms

- Classification (supervised learning)
  - Decision trees
  - Random Forest
  - Neural Networks
  - Support Vector Machines
  - k-nearest neighbour
  - Naïve Bayesian classifier

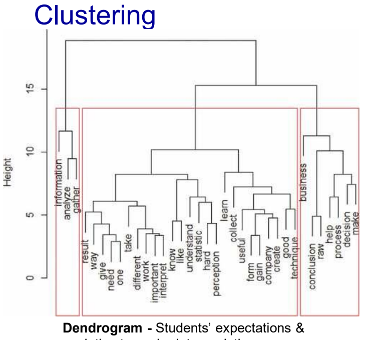


56

56

Text Mining Tasks:
Clustering

- **Organise** document collections
  - Automatically identify hierarchical / topical relation among documents



Dendrogram - Students' expectations & anxieties towards data analytics course
Bringula et.al. (2022)

57

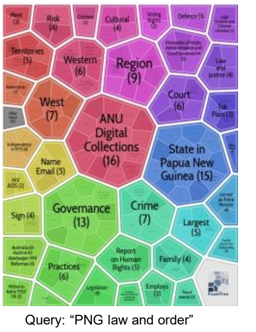
57

Text Mining Tasks:
Clustering

- **Grouping** search results
  - Organize documents by topics
  - Facilitate user browsing

https://search.carrot2.org/#/workbench

"Carrot² organizes search results into topics."
"With an instant overview of what's available, it is easier to look up."



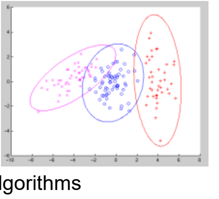
Query: "PNG law and order"

58

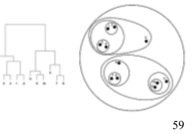
58

Text Mining Tasks:
Some Clustering Algorithms

- Typical **partitional clustering** algorithms
  - *k*-means clustering
  - Gaussian Mixture Model (GMM)
    - Consider variance within cluster also
  - Expected Maximization (EM)
    - Iterative refinement of cluster assignment



- Typical **hierarchical clustering** algorithms
  - Top-down divisive clustering
  - Bottom-up agglomerative clustering

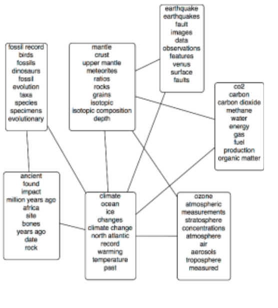


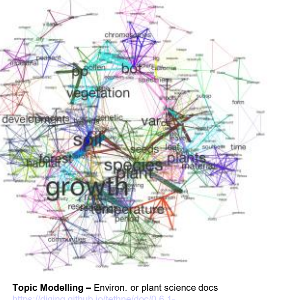
59

59

Text Mining Tasks:
Topic Modelling

- Topic modeling: Grouping words into topics





Topic Modelling – Environ. or plant science docs
https://diging.github.io/tethne/doc/0.6.1-beta/_images/semantic_network.png

60

60

Text Mining Tasks:
Natural Language Processing (NLP)

- **Objective**:
  - Enable machines to **understand**, **interpret**, and **generate** human language
- **Why NLP?**
  - Human language is **unstructured** and **ambiguous**; computers need structured input
- **Key tasks:**
  - Tokenisation: split text into words
  - POS (Parts-of-Speech) tagging: assign word classes e.g. nouns, verbs
  - NER (Named Entity Recognition): identify names, places, etc.
  - Parsing: analyse sentence structure
  - Sentiment: detect emotions/opinions
- **Applications**
  - Chatbots, translation, text summarisation, search, speech recognition

61

61

Towards Generative Text Mining:
from *Embeddings* to *Context*

- **Contextual Representations**
  - Mordern models represent words based on surrounding words
  - E.g. "*bank*" in "*bank account*" ≠ "*river bank*"
  - Representation changes with context
- **Large Language Models (LLMs)**
  - Learn from very large text corpora
  - Predict and understand text based on **context**
  - Used for:
    - Search, Question answering, Summarisation, Text generation
    - Do all or many task – move to AGI (Artificial General Intelligence)
- **Text mining evolving**:
  Move from ***word matching*** → ***meaning*** → ***context-aware meaning***

62

62

Classical text mining vs LLM-era text mining

- **Classical methods**:
  - BoW, TF-IDF, Naive Bayes, cosine, clustering
  - Remain useful as Simple, interpretable, data-efficient & computationally lighter
- **Modern methods**:
  - Embeddings, transformers, dense retrieval, re-ranking, RAG, LLM prompting
  - Improve semantic matching and generation, but need more compute and stronger evaluation
  - LLMs generate (most likely) text, token-by-token
  - Powerful, but not equivalent to understanding or truth.

63

63

Text mining KDD

1. **Domain Understanding** - What problem are we solving?
   - Define task: classification, similarity, retrieval, summarization, etc.
2. **Data Selection** - What text do we use?
   - Collect documents from **corpus** (e.g., feedback, emails, reports)
3. **Data Cleaning & Preprocessing - How to prepare the raw text?**
   - Tokenisation, stopword removal, normalisation, etc.
4. **Data Transformation - How do we convert text into features?**
   - Bag-of-Words, Boolean, TF-IDF, Vector Space Model, Embeddings
5. **Data Mining - What patterns or predictions do we learn?**
   - Apply text mining algorithms to perform task in (1)
6. **Evaluation & Interpretation - How good are the results?**
   - Accuracy, similarity scores, relevance, usefulness, etc.
7. **Knowledge Presentation & Use** - How is the result used?
   - Decision support, search systems, dashboards, AI tools

64

64

Evaluation:
What makes a Text Mining System ood?

- Examples of Evaluation Metrics:
  - ***Classification***: accuracy, precision, recall, F1
  - ***Clustering/topic discovery***: coherence, interpretability, usefulness
  - ***Retrieval***: precision@k, recall@k, MAP, NDCG
  - ***Generation***: faithfulness, relevance, factual grounding, safety
- Human judgment still matters
- Task-appropriate evaluation:
  - E.g. Retrieval and generation should not be evaluated like ordinary classification.

65

65

Text mining KDD

1. **Domain Understanding** - What problem are we solving?
   - Define task: classification, similarity, retrieval, summarization, etc.
2. **Data Selection** - What text do we use?
   - Collect documents from **corpus** (e.g., feedback, emails, reports)
3. **Data Cleaning & Preprocessing - How to prepare the raw text?**
   - Tokenisation, stopword removal, normalisation, etc.
4. **Data Transformation - How do we convert text into features?**
   - Bag-of-Words, Boolean, TF-IDF, Vector Space Model, Embeddings
5. **Data Mining - What patterns or predictions do we learn?**
   - Apply text mining algorithms to perform task in (1)
6. **Evaluation & Interpretation - How good are the results?**
   - Accuracy, similarity scores, relevance, usefulness, etc.
7. **Knowledge Presentation & Use - How is the result used?**
   - Decision support, search systems, dashboards, AI tools

66

66

Knowledge Presentation & Use

- How are results **communicated and used**?
  - Convert results into actionable insights
  - Present findings clearly to decision makers
- **Common Outputs**
  - *Dashboards* – trends in feedback, sentiment, topics
  - *Search systems* – ranked relevant documents
  - *Alerts* – fraud, complaints, risk keywords
  - *Reports* – summaries and recommendations
  - *AI tools* – chatbots, assistants, Q&A systems
- **Student Feedback Example**
  - Common praise: practical labs helpful; Common concern: lectures too fast; **Action**: adjust pacing and expand lab support
- **Good models create value only when results are understandable and useful.**

67

67

Responsible and Ethical Use of Text Mining & LLMs

- **Key Risks in Text Mining**
  - *Bias* in data or labels: unfair or distorted outcomes
  - *Privacy* and *confidentiality*: sensitive text may expose personal information
  - *Copyright* and *data use*: respect ownership, consent, and licensing
- **Additional Risks in LLMs**
  - *Hallucination*: confident but false outputs
  - *Misinformation*: inaccurate summaries or answers at scale
  - *Lack of transparency*: difficult to explain how outputs were produced
- **Good Practice**
  - Verify important outputs with human judgment
  - Use trusted and appropriate data sources
  - Protect personal or confidential information
  - Apply extra caution in high-stakes areas (health, law, finance, education)

Powerful text mining systems must be **accurate, fair, safe, & responsibly used**.

68

68

Summary

- Text mining extends data mining to unstructured text.
- Raw text must be transformed into structured representations.
- Common representations: Boolean, BoW, TF-IDF, embeddings.
- Cosine similarity compares vectors for retrieval and matching.
- Text mining supports classification, clustering, search, retrieval, summarisation, Q&A etc..
- Modern systems use context-aware models and LLMs, but classical methods remain useful.
- Effective systems also require evaluation, ethics, and practical use.

69

69